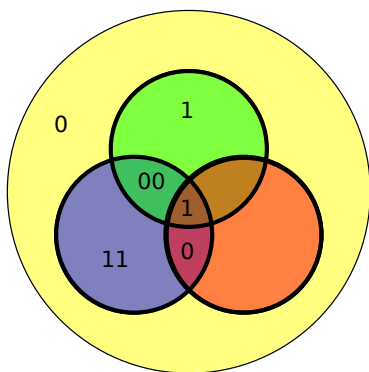


Hamming Codes: The Error-Correcting Codes

When bits are transmitted or sent over the computer network, they can easily get corrupted due to interference and network problems, and lead to errors. Hamming codes are linear codes used to detect and correct the errors that can occur when data is moved from the sender to the receiver. Let's take a look at their implementation in Python.



In 1950, R.W. Hamming created an error correction code called the Hamming code. It divides data into fixed length blocks, making it a block based error detection and correction algorithm. Extra or redundant parity bits are generated and embedded into the data message before transmission. After the receiver gets them, these parity bits are recalculated to detect and correct the error.

Up to 2 bit errors can be detected and up to 1 bit errors can be corrected. For example, in case of *even parity*, the number of 1s in specific positions should turn out to be even. This will ensure the integrity or correctness of the message. The procedure can be described in the following steps:

- Calculation of the number of redundant bits
- Finding the position of parity/redundant bits
- Filling the parity bits
- Sending the message
- On the receiver side, the steps are:
 - Receiving the message
 - Finding the position of parity/redundant bits
 - Comparing the parity bits to find the error

Algorithm overview

We are going to use *even parity* in this example.

The user has to send a data stream of bits from one endpoint to another. The number of bits in the data stream is represented as 'm'. And the hamming encoding algorithm is going to insert 'r' number of extra parity bits in the message for encoding before sending on the wire.

If 'm' is the number of data bits and 'r' is number of redundant bits, the following condition must be satisfied:

$$2^r \geq m + r + 1 \quad \dots \text{eq1}$$

The parity bits will be inserted in specific locations of the encoded data stream. These locations are powers of 2; e.g. ($2^0=1$, $2^1=2$, $2^2=4$, $2^3=8$, etc).

Let us take an example. Say we have a data stream of 4 bits; so, $m=4$. This is represented as follows:

D1	D2	D3	D4
----	----	----	----

We now need to calculate the number of parity bits:

If $r=1$ we put in eq1 $2^1 < 4+1+1$, so not ok

If $r=2$ we put in eq1 $2^2 < 4+2+1$, so not ok

If $r=3$ we put in eq1 $2^3 = 4+3+1$, this is ok

So the number of redundant bits is $r=3$. After the insertion of parity bits, the data stream will look like this:

P1	P2	D1	P3	D2	D3	D4
----	----	----	----	----	----	----

These parity bits are calculated from XOR of data bits at specific locations. The specific data bit positions are found based on the following table (where the corresponding bit is 1).

bit location	P3	P2	P1
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

So,

$$P1 = B1 \oplus B3 \oplus B5 \oplus B7$$

$$P2 = B2 \oplus B3 \oplus B6 \oplus B7$$